

Documentación Oficial de Jules

Módulo 4: Guías e Integraciones

Google Jules Agent Docs
Fuente: <https://jules.google/docs>

Tabla de Contenido

- [1. Continuous AI Overview](#)
- [2. Integrations](#)
- [3. Render Integration](#)

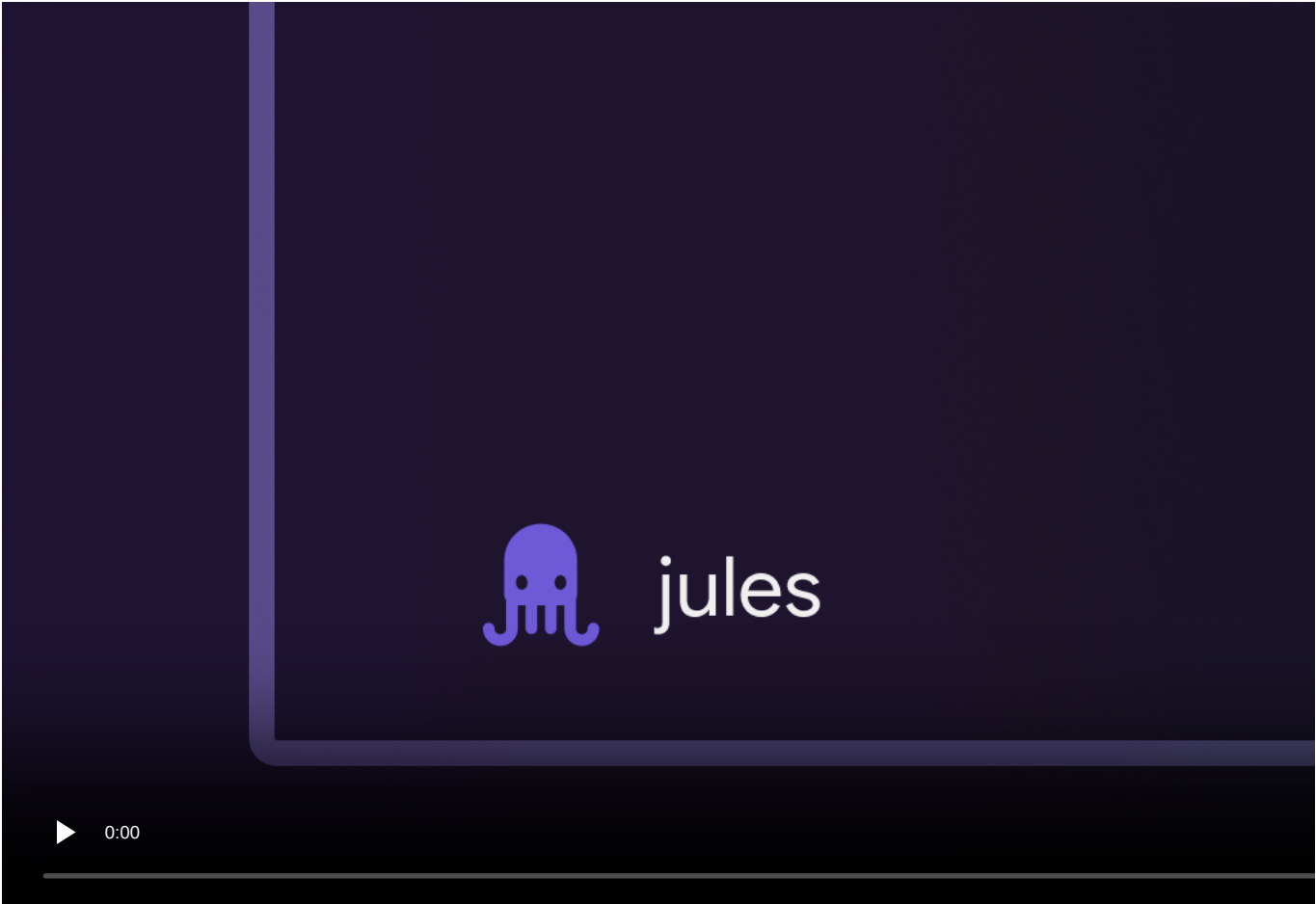
Continuous AI Overview

Fuente: <https://jules.google/docs/guides/continuous-ai-overview>

Continuous AI Overview

Launched

Continuous AI

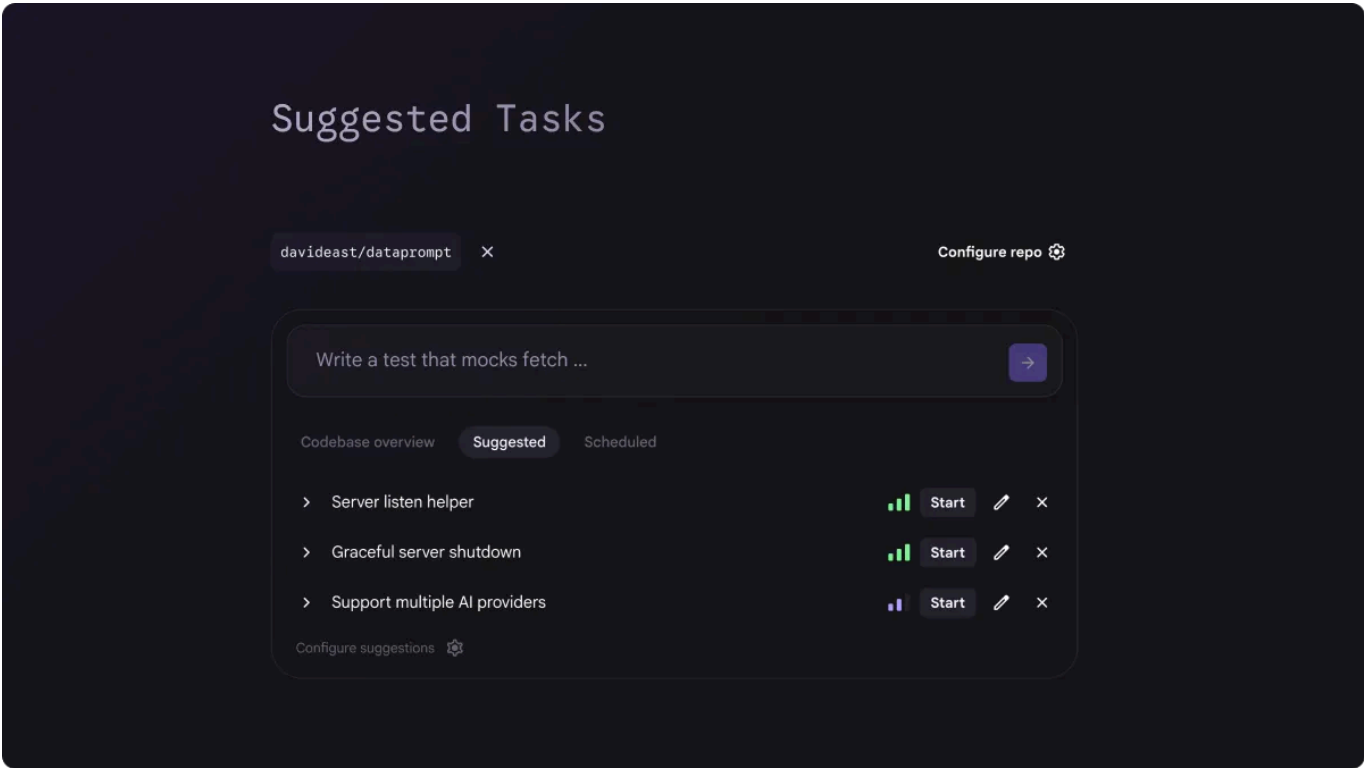


Last week, we shipped **Continuous AI** at Jules. This is a massive shift in how you’re going to work with Jules moving forward.

We’ve added three key features that turn Jules from a tool you use into a system that works for you. Here’s the breakdown:

Suggested Tasks

Stop hunting through your codebase for that one `// TODO` you forgot about.



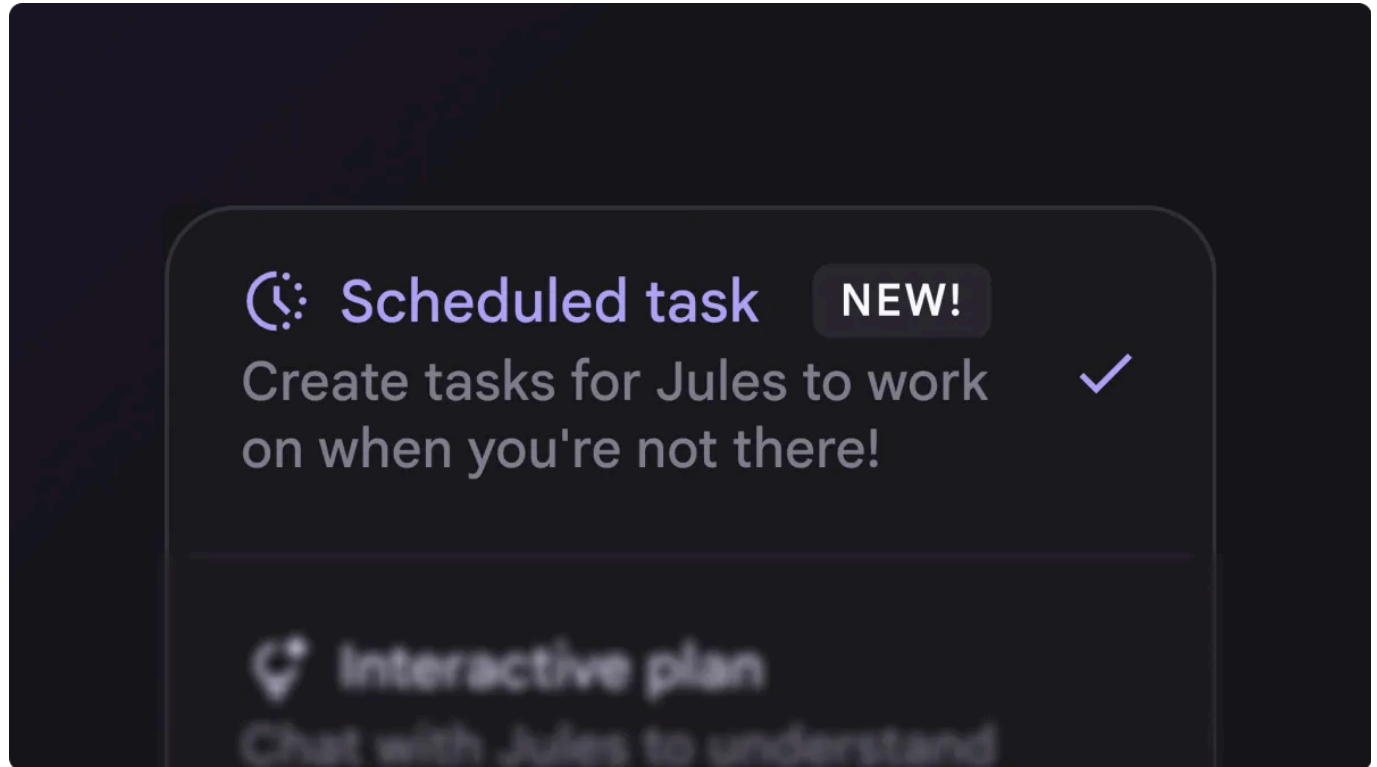
When you toggle the **Suggested Task** switch, Jules analyzes your entire codebase to find those floating comments and surfaces them as actionable tasks right in your dashboard.

Jules finds these tasks and analyzes them in context, providing a rationale for the change, and even giving you a confidence score on making that change independently. You can kick off multiple tasks in parallel and watch your to-do list shrink in real-time.

Pro-Tip: If you enable this and don't see any suggestions, it just means you're too organized! To get things moving, write a prompt asking Jules to analyze and raise specific inline to-dos. Once that's committed, Jules will surface those new tasks automatically.

Scheduled Tasks

This might be my personal favorite. **Scheduled Tasks** allow you to write a prompt and set it to run on a daily, weekly, or monthly interval.

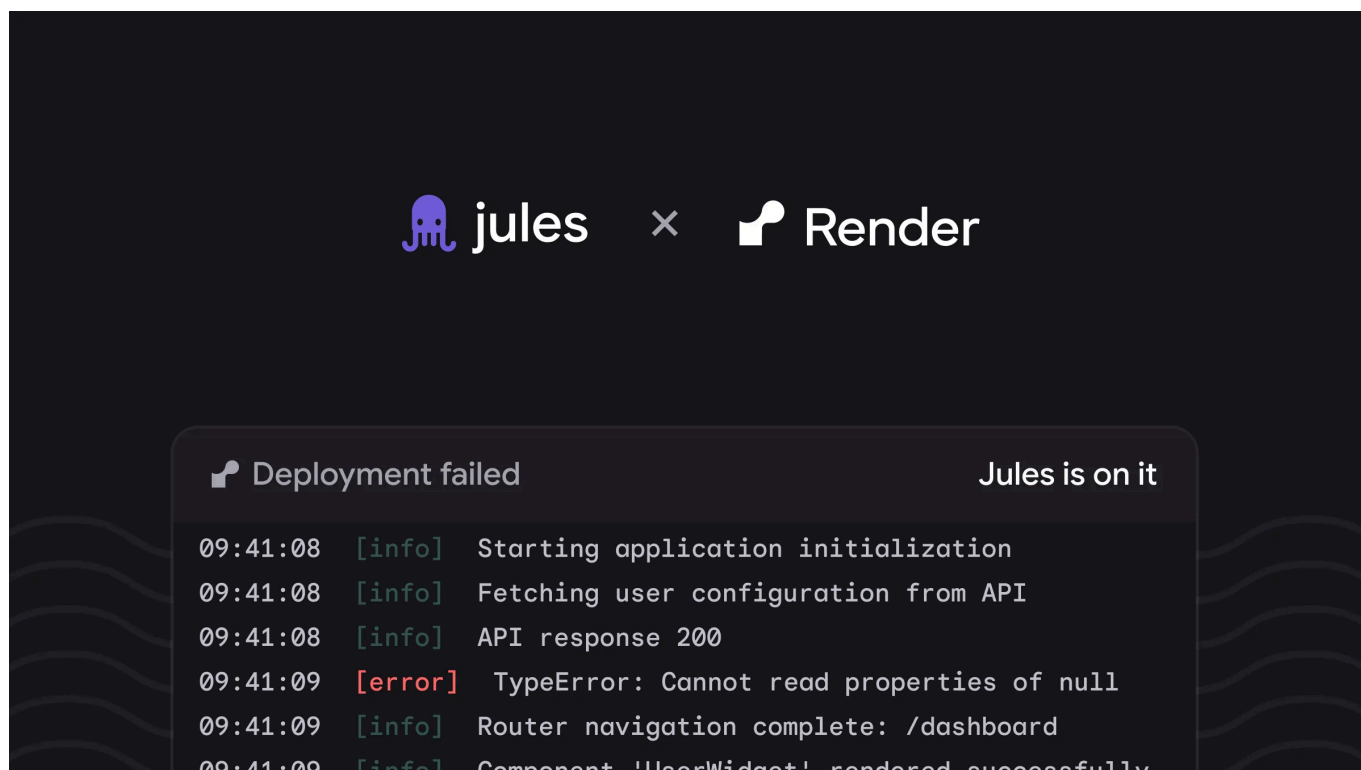


This unlocks some incredible use cases for CI/CD. We've all been there: a build breaks, and you have to drop everything to fix it. Now, you can run those builds on a schedule, and if something breaks, Jules can step in and fix it for you.

In the video you'll see that I scheduled a task that creates a nightly release to NPM for a library. In the past, when it's been broken, Jules caught it, figured out the fix, and published it under the nightly tag.

Render Integration

To close the loop on Continuous AI, you need a tight feedback loop. That's why we've integrated Jules with **Render**.



Render is a fantastic cloud platform for hosting web services, and it has a great integration with GitHub that creates preview deployments for PRs. But sometimes, those builds fail.

Now, when a build fails on Render, it automatically reports that failure back to Jules. Jules then analyzes the error, works on a fix, and sends a commit back to your PR—which triggers a fresh, successful redeploy. It's an automated loop that keeps your deployment pipeline moving without manual intervention.

The Big Picture

When you combine **Suggested Tasks**, **Scheduled Tasks**, and the **Render Integration**, you aren't just using an AI assistant anymore. You're building a continuous system for your apps, websites, and libraries.

We are incredibly excited to see how you use these new features to keep your code moving.

Integrations

Fuente: <https://jules.google/docs/integrations/>

Integrations

Jules works best when it has the same context you do. By connecting your external tools, you enable Jules to autonomously detect bugs, read build logs, and understand project requirements without manual input.

Deployment & CI/CD

[Section titled "Deployment & CI/CD"](#)

Connect your deployment pipelines to let Jules automatically detect build failures and propose fixes.

[Render](#) Automatically retrieve build logs and fix deployment failures. Jules watches for failed builds, analyzes the logs, and pushes fixes directly to Jules' PRs.

How Integrations Work

[Section titled "How Integrations Work"](#)

Jules uses **scoped access** to interact with your tools. When you connect an integration:

1. **Read-Only by default:** Unless otherwise specified, Jules requests the minimum permissions needed to read logs or status updates.
2. **Autonomous triggers:** Integrations allow Jules to wake up based on external events (like a failed web hook) rather than waiting for you to type a prompt.
3. **Secure storage:** API keys are encrypted and stored securely. They are never exposed in the chat interface or shared between sessions.

Render Integration

Fuente: <https://jules.google/docs/integrations/render>

Render Integration

Turn deployment failures into instant fixes. By connecting Jules to Render, you empower it to watch your builds, analyze errors, and push fixes before you even switch context.

Prerequisites

[Section titled "Prerequisites"](#)

Before generating your API key, ensure your Render and GitHub environments are ready:

1. **Enable Pull Request Previews:** Go to your Render dashboard and [enable Pull Request Previews](#).
2. **Verify GitHub Permissions:** Check your [GitHub Installations](#).
 - Locate **Google Labs Jules**.
 - If you see a **Review Request** link next to it, click it to accept the updated permissions.
 - If there is no link, your permissions are already up to date.

Setup

[Section titled "Setup"](#)

1. **Provision a Key** Navigate to the [Render Dashboard](#). Click the **Help** menu (top-right) and select **Coding Agents** to open the provisioning screen.

Alternatively, visit dashboard.render.com/jules directly.

2. **Create the Key** Click **Create API key**. Copy the key displayed.

This key is shown only once. If you lose it, you will need to provision a new one.

3. **Connect to Jules** In Jules, go to [Settings > Integrations](#). Paste the key into the Render field and submit.

Automated Fixes

[Section titled "Automated Fixes"](#)

Once connected, Jules automatically monitors the Pull Requests **it creates** for build failures.

1. **Jules creates a PR:** When you accept a plan, Jules opens a Pull Request with the changes.
2. **Detection:** If the Render build for that PR fails, Jules automatically analyzes the logs.
3. **Resolution:** Jules writes a fix and pushes a new commit to the same branch.
4. **Merge:** You review the fix just like any other change and merge when ready.

Jules currently only monitors and fixes build failures on Pull Requests created by Jules. It does not automatically debug failures on PRs created by you or other team members.